

# Графика и большой проект

Черепашка рисует на экране, программы запоминают данные в файлах. А в финале — своя «Змейка».

Четверть 3 · 9 уроков

## ЧТО В ЭТОЙ ЧАСТИ

1. **Урок 15.** Черепашка turtle: первые линии
2. **Урок 16.** Циклы и turtle: узоры
3. **Урок 17.** Функции и turtle: рисуем имя
4. **Урок 18.** Файлы: программа запоминает
5. **Урок 19.** 🖨️ Проект «Записная книжка»
6. **Урок 20.** Защита и разбор ошибок
7. **Урок 21.** Готовимся к «Змейке»
8. **Урок 22.** 🐍 Проект «Змейка»: начало
9. **Урок 23.** 🐍 «Змейка»: финал и защита

# Черепашка turtle: первые линии

## ЧЕМУ НАУЧИМСЯ

- Запускать графику модулем **turtle**
- Двигать черепашку: вперёд, поворот
- Нарисовать квадрат и треугольник



## Программа умеет рисовать

До сих пор наши программы работали с текстом и числами. Но Python умеет и **рисовать**. В нём есть встроенный модуль **turtle** — «черепашка». Она ползает по экрану и оставляет линию. Командуешь ей — получаешь рисунок.



**На пальцах.** Представь черепашку с кисточкой на спине. Куда она ползёт — там остаётся краска. Ты говоришь «вперёд на 100», «повернись направо» — и из этих команд складывается картинка.



## Разбираем код

Рисуем квадрат — четыре раза «вперёд и поворот»:

```
kvadrat.py

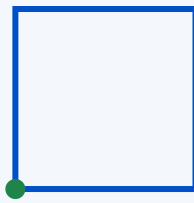
import turtle

t = turtle.Turtle()

t.forward(100)    # вперёд на 100
t.right(90)       # поворот направо на 90°
t.forward(100)
t.right(90)
t.forward(100)
t.right(90)
t.forward(100)

turtle.done()     # оставить окно открытым
```

НА ЭКРАНЕ ПОЯВИТСЯ



Команда `forward` ведёт черепашку вперёд, `right` поворачивает на угол. Четыре стороны по 100 с поворотами на  $90^\circ$  — получается квадрат. `turtle.done()` в конце не даёт окну закрыться сразу.

#### КОМАНДЫ ЧЕРЕПАШКИ

`forward(n)` / `backward(n)` — вперёд / назад. `right(угол)` / `left(угол)` — поворот.  
`t.color("red")` — цвет линии. Углы в градусах: полный круг —  $360^\circ$ .

#### 👉 ЗАДАНИЕ В ПАРЕ

##### Треугольник

Нарисуйте треугольник. Подсказка: три стороны, а поворачивать нужно на  $120^\circ$  (потому что  $360 \div 3 = 120$ ).

*Водитель за клавиатурой, штурман сверяется с учебником. Через 10 минут — меняетесь.*



#### Частые ошибки

##### ТАК НЕ РАБОТАЕТ

```
import turtle
forward(100)
```

# окно мгновенно закрылось

##### А ТАК ПРАВИЛЬНО

```
t = turtle.Turtle()
t.forward(100)
```

`turtle.done()` # в конце

Сначала создай черепашку `t = turtle.Turtle()`, потом командуй ей. И `turtle.done()` в конце, чтобы окно не закрылось.

#### 🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Нарисуй прямоугольник (стороны разной длины).
2. Нарисуй свои инициалы (буквы из прямых линий: Л, Т, Г).

★ **Со звёздочкой:** нарисуй домик — квадрат и треугольник-крышу.

# Циклы и turtle: рисуем узоры

## ЧЕМУ НАУЧИМСЯ

- Соединять цикл **for** с черепашкой
- Рисовать фигуры коротким кодом
- Создавать спирали и узоры



## Цикл экономит команды

Квадрат на прошлом уроке занял 7 строк. Но ведь мы повторяли одно и то же: «вперёд, поворот». А что повторяется — просится в **цикл**. Соединим то, что уже знаем: `for` из четверти 1 и черепашку.



**На пальцах.** Вместо того чтобы 4 раза писать «вперёд, направо», говоришь: «повтори 4 раза: вперёд, направо». Короче, понятнее, и легко поменять 4 на 5 — получится пятиугольник.



## Разбираем код

Квадрат в цикле — всего 3 строки вместо 7:

```
uzory.py

import turtle
t = turtle.Turtle()

# квадрат в цикле
for i in range(4):
    t.forward(100)
    t.right(90)

turtle.done()
```

Хочешь другую фигуру — поменяй число повторов и угол. Правило простое: **угол =  $360 \div \text{число сторон}$** . Для шестиугольника: 6 сторон, поворот 60°.

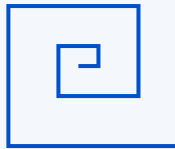
А вот узор-спираль — черепашка рисует и каждый раз чуть-чуть увеличивает шаг и угол:

```
spiral.py

t = turtle.Turtle()
for i in range(100):
```

```
t.forward(i)      # шаг растёт: 0,1,2,3...
t.right(91)       # чуть больше 90 – закрутка
turtle.done()
```

ПРИМЕРНО ТАК



### ФОРМУЛА ФИГУРЫ

Правильная фигура: `for i in range(число_сторон)`, внутри `forward(длина)` и `right(360 / число_сторон)`. Меняешь одно число — меняется фигура.

### 👉 ЗАДАНИЕ В ПАРЕ

#### Пятиугольник и звезда

Нарисуйте правильный пятиугольник (5 сторон, поворот  $72^\circ$ ). Потом попробуйте звезду: 5 повторов, но поворот  $144^\circ$ .

Через 10 минут поменяйтесь ролями.

### ⚠ Частые ошибки

#### ТАК НЕ РАБОТАЕТ

```
for i in range(4):
t.forward(100)
t.right(90)
```

#### А ТАК ПРАВИЛЬНО

```
for i in range(4):
    t.forward(100)
    t.right(90)
```

Обе команды должны быть с отступом — тогда они повторяются вместе. Без отступа цикл «не увидит» вторую строку.

### 🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Нарисуй шестиугольник и восьмиугольник (посчитай углы по формуле).
2. Нарисуй ряд из трёх квадратов (подсказка: между ними подними перо `t.penup()`).

★ **Со звёздочкой:** нарисуй «цветок» — повтори круг 12 раз, каждый раз поворачивая на  $30^\circ$ .



# Функции и turtle: рисуем имя

## ЧЕМУ НАУЧИМСЯ

- Выносить рисование фигуры в функцию
- Управлять пером: поднять и опустить
- Рисовать много фигур в разных местах



## Функция-художник

Соединяем ещё две знакомые вещи: **функции** из четверти 2 и черепашку. Опишем «как рисовать квадрат» один раз в функции — и будем рисовать квадраты где угодно одной командой.



**На пальцах.** Это как штамп. Вырезал штамп «квадрат» один раз — и ставишь его на бумаге сколько хочешь, в любом месте. Функция `kvadrat()` — такой штамп.



## Разбираем код

Функция рисует квадрат заданного размера, а перо помогает переходить без линии:

```
import turtle
t = turtle.Turtle()

def kvadrat(size):
    for i in range(4):
        t.forward(size)
        t.right(90)

kvadrat(50)           # маленький

t.penup()             # поднять перо — не рисуем
t.forward(120)
t.pendown()           # опустить — снова рисуем

kvadrat(80)           # большой, в новом месте

turtle.done()
```

`kvadrat(size)` рисует квадрат любого размера — передаём число при вызове.

## 2 penup / pendown — перо

Поднятое перо не оставляет линию. Так черепашка переходит в новое место, не соединяя фигуры чертой.

### УПРАВЛЕНИЕ ПЕРОМ

`penup()` — поднять (двигаемся без линии). `pendown()` — опустить (снова рисуем).  
Незаменимо, когда рисуешь несколько отдельных фигур: буквы имени, ряд домиков.

### 👉 ЗАДАНИЕ В ПАРЕ

#### Три квадрата в ряд

Используя функцию `kvadrat` и перо, нарисуйте три квадрата разного размера в ряд, не соединённые линиями.

*Через 10 минут поменяйтесь ролями.*



### Частые ошибки

#### ТАК НЕ РАБОТАЕТ

```
# перешёл, но забыл penup  
t.forward(120)
```

#### А ТАК ПРАВИЛЬНО

```
t.penup()  
t.forward(120)  
t.pendown()
```

Если не поднять перо, черепашка нарисует лишнюю линию между фигурами. Не забудь опустить перо обратно.

### 🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Напиши функцию `treugolnik(size)` и нарисуй два треугольника рядом.
2. Нарисуй первую букву своего имени из линий и фигур.

★ **Со звёздочкой:** напиши функцию, рисующую квадрат заданного цвета — цвет передавай аргументом.



# Файлы: программа запоминает

## ЧЕМУ НАУЧИМСЯ

- Сохранять данные в файл .txt
- Читать данные из файла обратно
- Понять, почему это важно для настоящих программ



## Проблема забывчивости

Все наши программы **забывали** всё, как только закрывались. Ввёл данные, программа посчитала — закрыл, и всё пропало. Настоящие программы так не делают: заметки, игры, контакты сохраняются. Секрет — в **файлах**.



**На пальцах.** Память программы — как разговор: закончился и забылся. Файл — как записная книжка: закрыл, убрал, а завтра открыл — всё на месте. Файл переживает закрытие программы.



## Записываем в файл

```
zapis.py

# "w" — режим записи (write)
file = open("zametki.txt", "w")
file.write("Купить хлеб\n")
file.write("Позвонить бабушке\n")
file.close()

print("Заметки сохранены!")
```

`open` открывает файл, `"w"` означает «для записи». `write` записывает строку, а `\n` — это переход на новую строку. `close` обязательно закрывает файл — как закрыть кран.



## Читаем из файла

```
chtenie.py

# "r" — режим чтения (read)
file = open("zametki.txt", "r")
text = file.read()
```

```
file.close()
```

```
print(text)
```

НА ЭКРАНЕ ПОЯВИТСЯ

Купить хлеб

Позвонить бабушке

### ВАЖНО ЗАПОМНИТЬ

`open(имя, "w")` — записать (стирает старое!). `open(имя, "r")` — прочитать. `write` пишет, `read` читает, `close` закрывает. `\n` — новая строка. Есть ещё `"a"` — дописать в конец, не стирая.

### 👉 ЗАДАНИЕ В ПАРЕ

#### Дневник

Программа спрашивает, как прошёл день, и сохраняет ответ в файл `dnevnik.txt`. Затем отдельная программа читает файл и показывает запись.

*Через 10 минут поменяйтесь ролями.*



### Частые ошибки

#### ТАК НЕ РАБОТАЕТ

```
file = open("f.txt", "w")
file.write("текст")
# забыли close
```

```
write("привет")\n # это не перенос
```

#### А ТАК ПРАВИЛЬНО

```
file = open("f.txt", "w")
file.write("текст")
file.close()
```

```
write("привет\n")
```

Всегда закрывай файл через `close`. И помни: режим `"w"` стирает старое содержимое — для дописывания используй `"a"`.

### 🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Сохрани в файл список из трёх любимых фильмов, потом прочитай и выведи.
2. Программа дописывает (режим `"a"`) новую строку в файл при каждом запуске.

★ **Со звёздочкой:** счётчик запусков — программа читает число из файла, увеличивает и записывает обратно.



# «Записная книжка»

## ЧТО ПОСТРОИМ

- Программу, которая хранит контакты между запусками
- Соединим словари, циклы и файлы
- Меню с выбором действия



## Первая полезная программа

«Угадай число» и «Виселица» были играми. А это — **полезная** программа, которой можно реально пользоваться: добавлять контакты, искать их, и всё сохраняется в файл. Здесь соберётся много знакомого: словари, циклы, условия, файлы.



## Собираем книжку

```
knizhka.py

# читаем контакты из файла в начале
contacts = {}
try:
    file = open("contacts.txt", "r")
    for line in file:
        name, phone = line.strip().split(",")
        contacts[name] = phone
    file.close()
except:
    pass # файла ещё нет — не страшно

while True:
    print("1-добавить 2-найти 3-выход")
    choice = input("Выбор: ")

    if choice == "1":
        name = input("Имя: ")
        phone = input("Телефон: ")
        contacts[name] = phone
    elif choice == "2":
        name = input("Кого ищем? ")
        print(contacts.get(name, "Не найдено"))
    else:
        break
```

```
# сохраняем всё обратно в файл
file = open("contacts.txt", "w")
for name, phone in contacts.items():
    file.write(name + "," + phone + "\n")
file.close()
```

### 1 Загрузка при старте

В начале читаем файл и складываем контакты в словарь. `split(",")` разделяет строку «имя, телефон» на две части.

### 2 Меню в цикле

Цикл показывает меню снова и снова, пока не выберешь выход. Знакомый приём из «Угадай число».

### 3 Сохранение при выходе

Перед выходом записываем весь словарь обратно в файл — чтобы контакты не потерялись.

#### ЗАДАНИЕ В ПАРЕ

#### Соберите и проверьте

Наберите программу. Добавьте пару контактов, выйдите, запустите снова — контакты на месте? Значит, файл работает!

*Разбирайте по частям: сначала меню, потом файл.*

#### ДОМАШНЕЕ ЗАДАНИЕ

1. Добавь в меню пункт «показать все контакты».
2. Сделай, чтобы при поиске несуществующего имени программа предлагала добавить его.

---

★ **Со звёздочкой:** добавь удаление контакта по имени.

# Защита проекта и разбор ошибок

## ЧЕМУ НАУЧИМСЯ

- Читать сообщения об ошибках и понимать их
- Находить и исправлять чужие ошибки
- Рассказывать о своей программе на защите



## Ошибка — это подсказка

За три четверти вы много раз видели красные сообщения об ошибках. Пора научиться **читать** их спокойно. Python всегда подсказывает, что не так и в какой строке — надо просто присмотреться.



**На пальцах.** Ошибка — не «ты плохой программист», а навигатор, который говорит «поверни, ты не туда». Опытные программисты ошибаются постоянно — просто быстро читают подсказку и правят.



## Три частые ошибки в лицо

```
oшибki.py

# SyntaxError – забыл двоеточие или скобку
if x > 5
    print("много")

# NameError – имя не создано (опечатка)
print(nmae)

# IndentationError – неправильный отступ
for i in range(3):
print(i)
```

## ЧИТАЕМ ОШИБКУ

**SyntaxError** — синтаксис: забыл двоеточие, скобку, кавычку. **NameError** — имя не существует, чаще опечатка. **IndentationError** — отступы. **TypeError** — смешал текст и число. Последняя строка ошибки — самая важная.



ЗАДАНИЕ В ПАРЕ

## Охота на баги

Учитель даст программу с 3–4 ошибками. Ваша задача — запустить, прочитать каждую ошибку, исправить и добиться, чтобы программа заработала. Кто быстрее?

*Читайте ошибки вслух друг другу — так быстрее находится причина.*



## Как защищать проект

Защита — это 2 минуты у проектора. Расскажи по простому плану:

### 1 Что делает программа

Одним предложением: «это записная книжка, хранит контакты в файле».

### 2 Как она работает

Покажи главное: «меню в цикле, контакты в словаре, сохранение в файл».

### 3 Что было сложно

Честно расскажи, где застрял и как решил. Это ценят больше всего.



## ДОМАШНЕЕ ЗАДАНИЕ

1. Подготовь защиту «Записной книжки» по плану из трёх пунктов.
2. Найди в своём старом коде любую ошибку, которую делал, и запиши, как её узнать.

★ **Со звёздочкой:** составь свою «шпаргалку ошибок» — 5 самых частых и как их чинить.

# Готовимся к «Змейке»: разбор идеи

## ЧЕМУ НАУЧИМСЯ

- Разбирать большой проект на части до начала кода
- Понимать, из чего состоит игра «Змейка»
- Планировать — главный навык программиста



## Сначала думаем, потом кодим

«Змейка» — самый большой проект года. Бросаться писать код сразу — плохая идея. Сначала **разберём игру на части**: что в ней есть, что за чем происходит. Это называется планированием, и это отличает хорошего программиста.



**На пальцах.** Строитель не начинает класть кирпичи наугад — сначала чертёж. Наш чертёж «Змейки»: понять, из каких кусочков состоит игра, до того как писать первую строку.



## Из чего состоит «Змейка»

Разберём игру на понятные части:

### 1 Змейка — это список

Тело змейки — список координат. Голова — первый элемент. Двигается — добавляем клетку спереди, убираем сзади.

### 2 Еда — случайная точка

Яблоко появляется в случайном месте (знакомый `random`). Съела — растёт, появляется новое.

### 3 Управление — клавиши

`turtle` умеет ловить нажатия стрелок и менять направление движения.

### 4 Проигрыш — врезалась

Если голова столкнулась со стеной или своим хвостом — игра окончена.

## ЧТО МЫ УЖЕ УМЕЕМ ДЛЯ «ЗМЕЙКИ»

Списки (тело змейки), циклы (главный цикл игры), функции (движение, проверки), `random` (еда), условия (столкновения), `turtle` (рисование). Вся игра — это сборка того, что вы уже знаете!

## ЗАДАНИЕ В ПАРЕ

### Нарисуйте план на бумаге

Не за компьютером, а на листе: нарисуйте, как выглядит игра, и запишите словами, что происходит на каждом шаге (голова двигается → проверяем еду → проверяем столкновение). Это ваш чертёж.

*Обсуждайте вслух — так рождается понимание.*

## ДОМАШНЕЕ ЗАДАНИЕ

1. Запиши по шагам, что происходит за один «тик» игры (одно движение змейки).
2. Подумай: где в «Змейке» пригодится список, а где словарь? Обоснуй.

---

★ **Со звёздочкой:** придумай, как добавить в игру подсчёт очков, и запиши идею.



 «Змейка»:  
начало

## ЧТО ПОСТРОИМ СЕГОДНЯ

- Окно игры и змейку из одного сегмента
- Движение по нажатию стрелок
- Основу главного цикла игры

 Собираем каркас

По нашему плану строим постепенно. Сегодня — **окно, змейка и управление**. На следующем уроке добавим еду, рост и проигрыш. Не пугайся длины кода — мы разберём его по кусочкам, и каждый кусочек знакомый.

 Каркас игры

```
import turtle

screen = turtle.Screen()
screen.setup(400, 400)
screen.title("Змейка")

# голова змейки
head = turtle.Turtle()
head.shape("square")
head.color("green")
head.penup()

direction = "stop"

def go_up():
    global direction
    direction = "up"

def go_down():
    global direction
    direction = "down"

# ловим нажатия стрелок
screen.listen()
```

```
screen.onkey(go_up, "Up")
screen.onkey(go_down, "Down")

# главный цикл игры
while True:
    screen.update()
    if direction == "up":
        head.sety(head.ycor() + 20)
    if direction == "down":
        head.sety(head.ycor() - 20)
```

## 1 Окно и голова

`Screen` создаёт окно игры. Голова — черепашка в форме квадрата. `penup`, чтобы не рисовала линию.

## 2 Функции направления

Каждая клавиша меняет переменную `direction`. Слово `global` позволяет функции менять переменную снаружи.

## 3 Главный цикл

`while True` — сердце игры. Каждый повтор: обновить экран и подвинуть голову в текущем направлении.

### 👉 ЗАДАНИЕ В ПАРЕ

#### Запустите и подвигайте

Наберите каркас и добавьте функции `go_left` и `go_right` по образцу (меняют `x` через `setx`). Запустите — квадрат должен двигаться стрелками.

На следующем уроке добавим еду и проигрыш.

### 🏠 ДОМАШНЕЕ ЗАДАНИЕ

1. Допиши движение влево и вправо, чтобы змейка слушалась всех четырёх стрелок.
2. Поменяй цвет и размер окна на свой вкус.

★ **Со звёздочкой:** сделай, чтобы змейка не могла развернуться на 180° (вверх нельзя, если едет вниз).



# «Змейка»: финал и защита

## ЧТО ДОДЕЛАЕМ

- Еду, которую змейка ест
- Проигрыш при столкновении со стеной
- Готовую игру для портфолио — и защиту



## Добавляем еду и проигрыш

К каркасу с прошлого урока добавляем еду и проверку столкновений. Показываю ключевые части — вставьте их в главный цикл:

```
snake_final.py

import random

# создаём еду (яблоко)
food = turtle.Turtle()
food.shape("circle")
food.color("red")
food.penup()
food.goto(0, 100)

score = 0

# --- эти проверки идут ВНУТРИ главного цикла ---

# съели еду?
if head.distance(food) < 20:
    x = random.randint(-180, 180)
    y = random.randint(-180, 180)
    food.goto(x, y)
    score = score + 1
    print("Очки:", score)

# врезались в стену?
if abs(head.xcor()) > 190 or abs(head.ycor()) > 190:
    print("Игра окончена! Очки:", score)
    break
```

Отдельная черепашка в форме круга. `head.distance(food)` измеряет расстояние до неё.

## 2 Съели — новая еда и очко

Если голова близко к еде — переносим еду в случайное место и добавляем очко.

## 3 Стена — конец игры

`abs()` берёт число без знака. Ушли за 190 по любой оси — `break`, игра окончена.



### ГЛАВНЫЙ ПРОЕКТ ГОДА

#### Доделайте и защитите «Змейку»

Соберите игру целиком. Сделайте её своей: свой цвет, счёт на экране, ускорение. Это ваш лучший проект — сохраните его в портфолио. На защите (3 минуты) покажите игру в действии и расскажите, как она устроена.

*Оба должны уметь объяснить главный цикл и проверку столкновений.*



#### ЧТО ПОКАЗАТЬ НА ЗАЩИТЕ

1. Змейка двигается стрелками, ест еду, растёт счёт.
2. Игра заканчивается при столкновении со стеной.
3. Вы добавили минимум одно своё улучшение.

★ **Со звёздочкой:** сделай, чтобы змейка росла (хвост из сегментов) и проигрывала, врезавшись в себя.